

SOFTWARE DESIGN MODELS

A Freshness Aware Agricultural Marketplace (AgriConnect)

1. Software Architecture Diagram.

AgriConnect follows a Client-Server architectural style with four distinct layers. This style was selected because it cleanly separates the user interface from the business logic and from the data store, while allowing external services to be integrated at the backend layer without exposing them to the client. The on-device AI component introduces a hybrid element. The TFLite model operates entirely offline within the client layer, which means the client layer itself contains both an online component and an offline component. This is a deliberate architectural decision driven by the offline scanning requirement for rural farmers with limited connectivity.

1.1 Client Layer

The client layer contains two distinct components:

- TFLite Model (Offline): A MobileNetV2 model embedded in the Flutter app that analyses 224×224 RGB images captured by the device camera. It runs entirely on the device to estimate freshness (0–100%) and remaining shelf life in days, without sending any data to the backend.
- Flutter Application (Online): The main application with role-based access, featuring authentication, farmer tools for produce scanning and listings, buyer tools for browsing and purchasing, driver tools for delivery management and a web-based admin dashboard for system administration.

1.2 Backend Layer

- Backend:: Hosted on Render with GitHub Actions for automated deployment. It powers the application's core services through seven main components:
- API Gateway: Validates user authentication, determines user roles, and routes every request to the appropriate service.
- Authentication Module: Manages OTP verification, authentication issuance and refresh, and activates user accounts after successful verification.
- Listing & Marketplace Module: Creates produce listings, generates QR codes, supports marketplace browsing with filtering and sorting and verifies QR codes.
- Price Recommendation Engine: Uses current The Ministry of Food and Agriculture(MOFA) reference prices and produces freshness score to recommend a selling price and a minimum acceptable price.
- Driver Dispatch Module: Matches deliveries to suitable drivers based on truck capacity and proximity, then manages job notifications and automatic reassignment if a driver declines.

- **Payment & Escrow Module:** Processes buyer payments through Paystack, securely holds funds, and releases payment to the farmer after successful delivery confirmation.
- **Admin Dashboard API:** Provides administrators with tools for user management, transaction monitoring, audit logging, price updates, and dispute resolution.

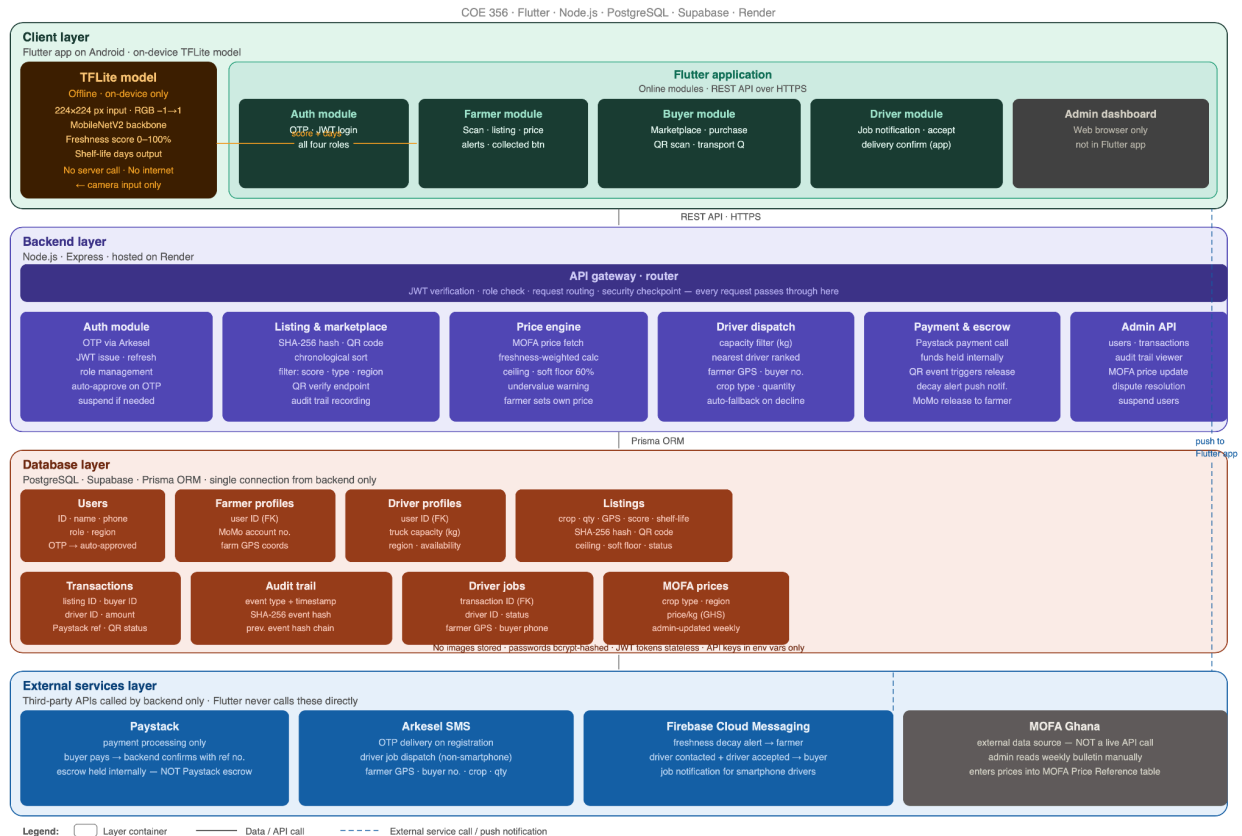
1.3 Database Layer

PostgreSQL hosted on Supabase, accessed exclusively through Prisma ORM from the backend. No other system component connects to the database directly. Contains eight tables: Users, Farmer Profiles, Driver Profiles, Listings, Transactions, Audit Trail, Driver Jobs and MOFA price reference.

1.4 External Services Layer

Four external services called by the backend only - the Flutter app never calls these directly:

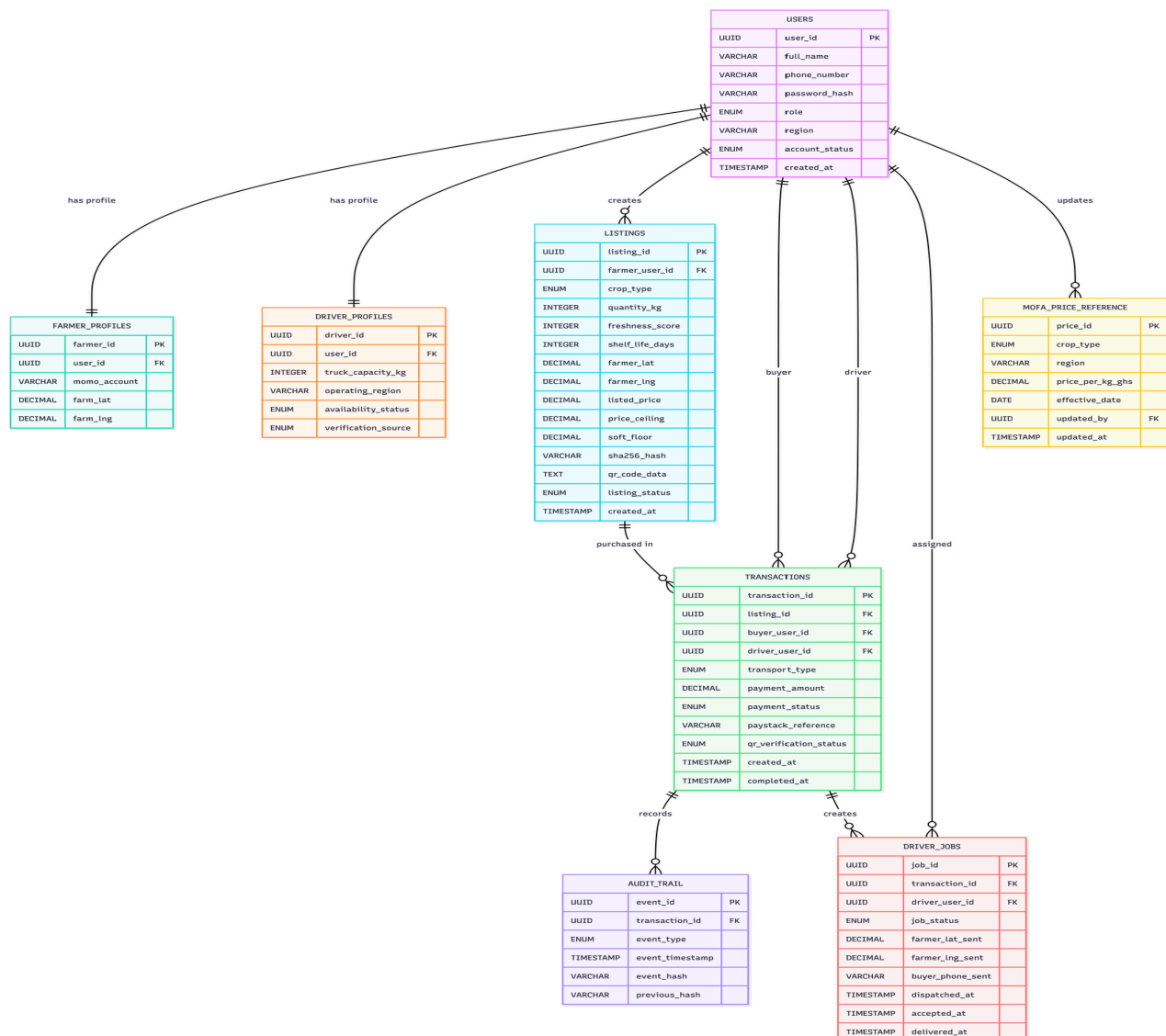
- **Paystack:** Processes buyer payments and securely holds funds in escrow until delivery is confirmed.
- **Arkesel SMS:** Sends OTPs during registration and SMS job notifications to drivers without smartphones.
- **Firestore Cloud Messaging (FCM):** Delivers push notifications to the Flutter app, including freshness alerts, driver updates and delivery job notifications.
- **MOFA Ghana:** Provides weekly market price bulletins, which administrators manually enter into the system through the Admin Dashboard to keep price recommendations up to date.



2. Entity Relationship(ER) Diagram.

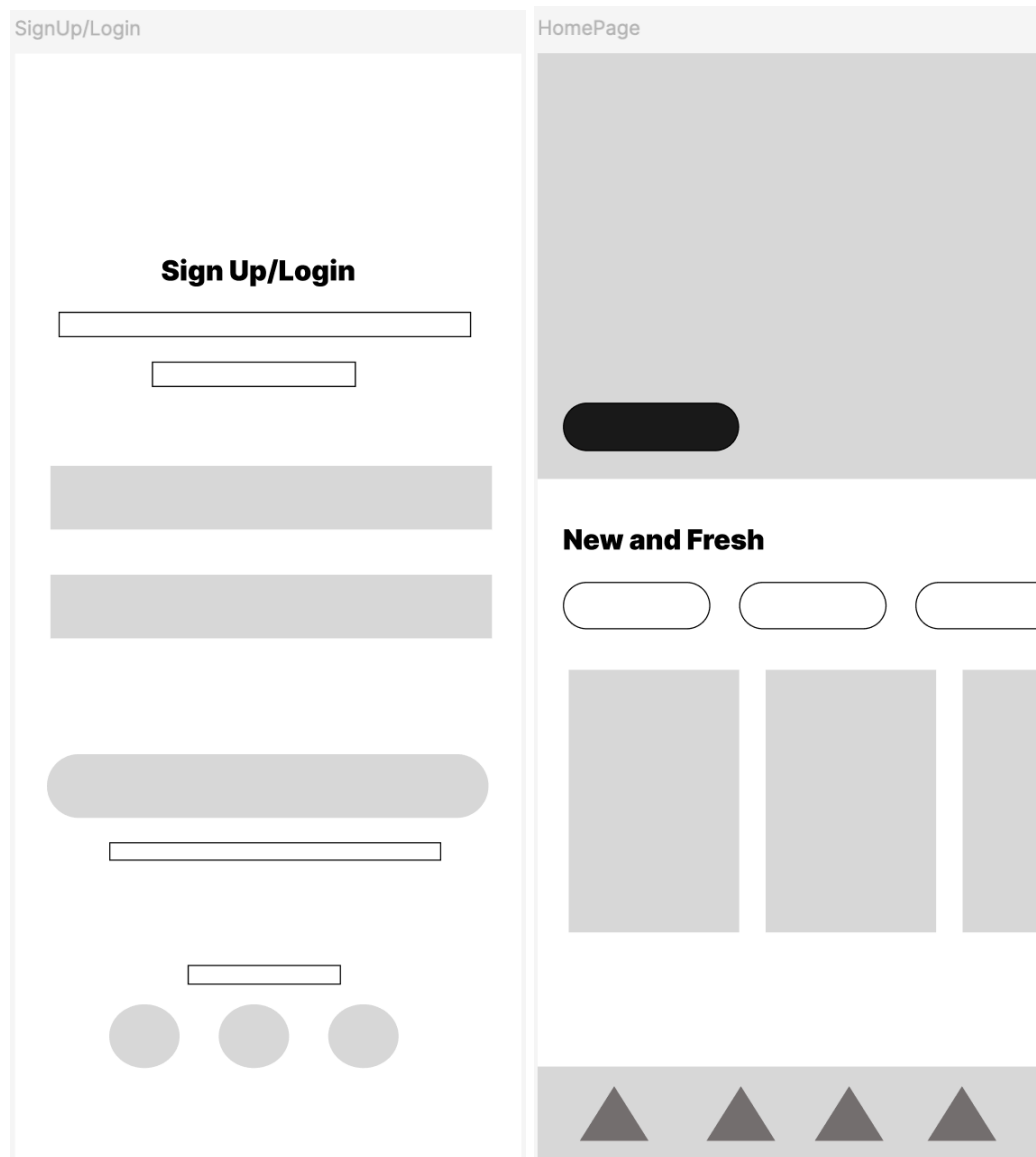
The AgriConnect database is designed using a relational model that separates different aspects of the system into independent but connected entities. At the centre of the database is the Users table, which stores the identity and authentication information for every user. Other entities reference this table through foreign keys to support role-specific functionality for farmers, buyers, drivers, and administrators while avoiding data duplication making it well normalized.

The remaining entities manage the core operations of the platform, including product listings, transactions, driver assignments, audit records, and MOFA price references. Relationships between these entities ensure data consistency and maintain referential integrity throughout the system. The ER diagram below illustrates the entities, their primary and foreign keys, and the cardinality of the relationships.



3. UI Wireframes

The wireframes illustrate the initial user experience of the AgriConnect mobile application. They present the authentication flow through the login screen and the application's home screen, where users can access key features such as freshness scanning, marketplace navigation, and other role-specific services.

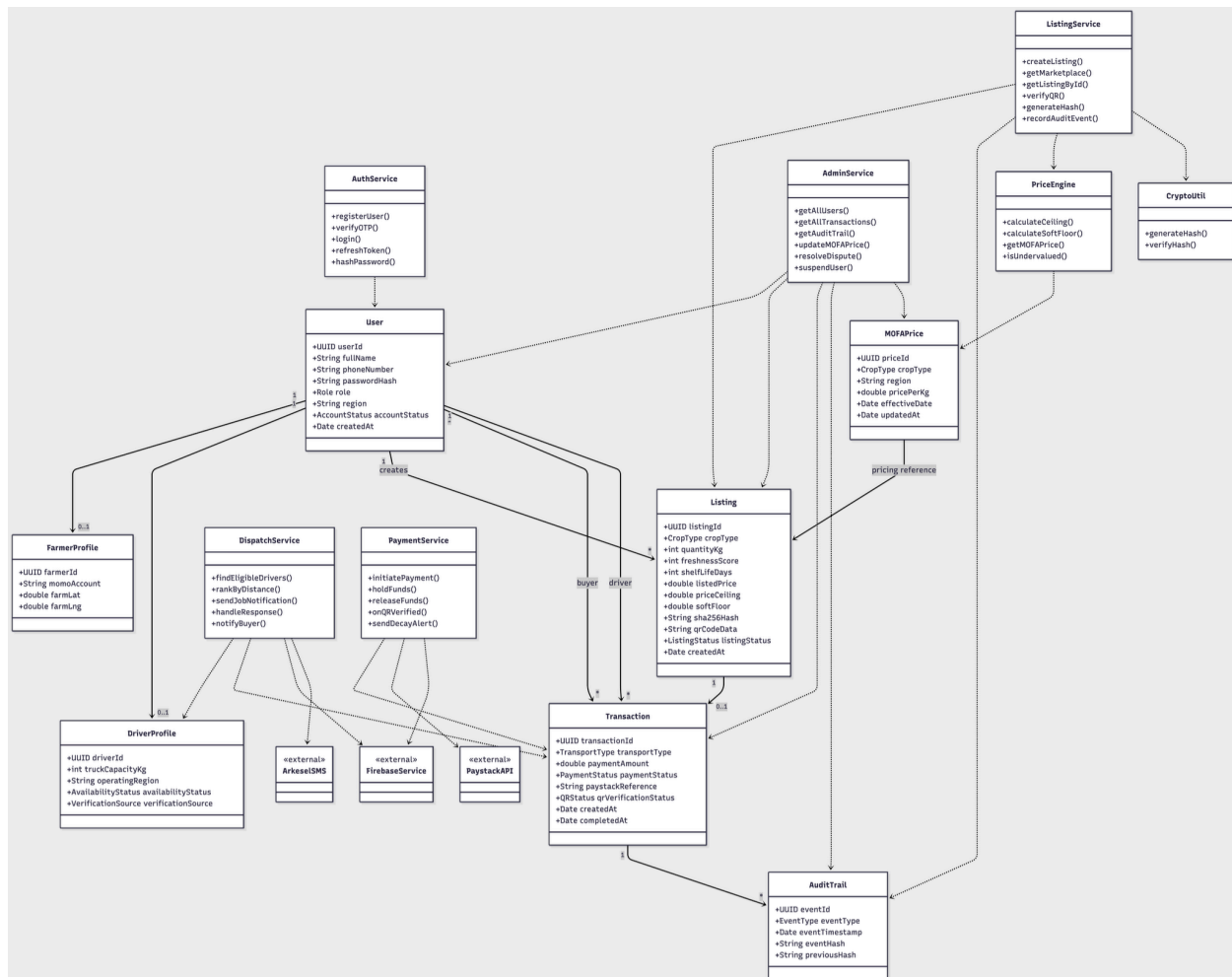


This link below leads to additional wireframes.

<https://www.figma.com/design/zX61MIItfQdAS9SU6PKcLb/AgriConnect-Wireframes?node-id=0-1&p=f>

4. UML Class Diagram

The UML class diagram illustrates the object-oriented design of the AgriConnect backend. It shows the system's core model classes, service classes, and their relationships. Model classes represent the application's primary data entities, while service classes make up the business logic responsible for authentication, listing management, pricing, payment processing, driver dispatch, and administration. The diagram also highlights dependencies on external services such as Paystack, Firebase Cloud Messaging, and Arkesel SMS, providing a high-level view of how the system components collaborate to deliver the platform's functionality.



5. Model Description

AgriConnect is made up of five modules. Which includes the farmer module, buyer module, transport module, trust and audit module, and administration module.

The farmer module enables farmers to assess produce freshness using on-device AI, create verified listings, receive price recommendations based on produce quality, monitor freshness over time, and automatically receive Mobile Money payments after successful delivery.

The Buyer Module allows buyers to browse AI-verified product listings, make secure purchases, choose between self-collection or driver delivery, and confirm delivery through QR code verification.

The Transport Module manages produce delivery by matching buyers with suitable drivers, handling delivery notifications, and confirming successful deliveries before payment is released.

The Trust and Audit Trail Module maintains the integrity of the platform by securing transaction records with cryptographic hashes and verifying product authenticity through QR codes.

The Administration Module provides administrators with tools to manage users, monitor platform activities, resolve disputes, and maintain regional crop prices used by the pricing engine.